

Obsługa wyjątków w języku Java

Zbigniew Marszałek

Obsługa wyjątków

Wszystkie działające aplikacje mogą napotkać sytuację, która powoduje błąd w jej działaniu. Sytuacja taka często wymaga interakcji z systemem operacyjnym i uzyskania informacji przez aplikację co spowodowało błąd. Przykładowo dla wprowadzonych danych przez użytkownika algorytm dokonuje dzielenia przez zero, próby otwarcia nieistniejącego pliku lub zapisu pliku w obszarze chronionym przez system. W języku Java znajdujemy technikę umożliwiającą obsługę wyjątków (ang. *Exception Handling*) i zapewniającą właściwą reakcję na zaistniałe błędy. W języku Java mówimy, że obiekt jest zgłaszany lub rzucony (ang. *thrown*) z jednego miejsca programu i przechwytywany w innej części programu (ang. *catches*).

Typy wyjątków

Wszystkie wyjątki są podklasami klasy *Throwable*. Poniżej klasy *Throwable* znajdują się dwie podklasy dzielące wyjątki na dwie kategorie. Podklasa *RuntimeException* obsługuje wyjątki w czasie działania programu takie jak dzielenie przez zero lub użycie nieprawidłowego indeksu tablicy. Druga podklasa *Error* obsługuje wyjątki, które nie są spodziewane w czasie prawidłowego działania programu. Są to wyjątki związane z samym środowiskiem języka Java, takie jak, przepełnienie stosu. Własne wyjątki powinny dziedziczyć po klasie *Exception*. **Klas**

Metoda	Opis
Throwable fileInStackTrace()	Zwraca obiekt Throwable zawierający pełny stos wywołań. Obiekt ten można ponownie zgłosić jako wyjątek.
Throwable getCause()	Zwracam wyjątek, który poprzedził aktualny wyjątek. W przypadku nie istnienia takiego wyjątku zwraca null.
String getLocalizedMessage()	Zwraca opis wyjątku po dokonaniu lokalizacji.
String getMessage()	Zwraca opis wyjątku.
StackTraceElement[] getStackTrace()	Zwraca tablicę zawierającą stos wywołań.

Metoda	Opis
<code>void printStackTrace()</code>	Wyświetla stos wywołań.
<code>void printStackTrace(PrintStream <i>stream</i>)</code>	Wysyła stos wywołań do wybranego strumienia.
<code>void printStackTrace(PrintWriter <i>stream</i>)</code>	Wysyła stos wywołań do wybranego strumienia.
<code>Throwable initCause(Throwable <i>causeExc</i>)</code>	Wiąże <i>causeExc</i> jako powód zgłaszanego wyjątku. Zwraca referencję do wyjątku.
<code>String toString()</code>	Zwraca obiekt String zawierający opis wyjątku
<code>void setStackTrace(StackTraceElement <i>elements</i>[])</code>	Ustawia stos wywołań na elementy przekazane w <i>elements</i> . Nie używany.

Kod obsługi wyjątku try – catch – finally

W języku Java możemy używać instrukcji *try* do wyznaczenia kodu, który może spowodować zgłoszenie wyjątku (ang. *guarded region*). Dla każdego bloku *try* musi być zdefiniowany przynajmniej jeden blok *catch* lub *finally*. Po przechwyceniu wyjątku przez system operacyjny, następuje próba odnalezienia bloku *catch*, który mógłby obsłużyć dany wyjątek. Przeszukiwana jest cała sekwencja bloków *catch* i system próbuje odnaleźć obsługę danego typu wyjątku przypisanego do instrukcji *try*. W przypadku nie odnalezienia obsługi zgłoszonego wyjątku (ang. *unhandled exception*) system zakończy działanie aplikacji.

Przedstawmy działanie instrukcji *try* na przykładzie dzielenia liczb całkowitych wprowadzonych przez użytkownika. Potencjalnie może nastąpić zgłoszenie dwóch wyjątków: liczba została wprowadzona nie prawidłowo lub nastąpiło podzielenie przez zero. Sprawdźmy działanie przedstawionego programu poniżej:

```
Scanner in = new Scanner(System.in);
try
// Instrukcja umożliwiająca wyznaczenie kodu
// mogącego doprowadzić do zgłoszenia wyjątku.
{ System.out.print
    ("Proszę podać pierwszą liczbę:");
    int a= in.nextInt();
    System.out.print
        ("Proszę podać drugą liczbę:");
    int b= in.nextInt();
// Wczytanie dwóch liczb całkowitych.
    int c = a/b;
// Dzielenie liczb całkowitych.
    System.out.printf("%d/%d=%d",a,b,c);
// Wypisanie wyników dzielenia.
}
```

Przechwycenie wyjątków

```
    catch (ArithmeticException ex) {  
// Blok przechwytyjący wyjątek.  
    System.out.println(ex.getMessage()); }  
// Wypisanie informacji o zgłoszonym wyjątku.  
    catch (Exception ex) {  
// Blok przechwytyjący wyjątek.  
    System.out.println(ex.getMessage()); }  
// Wypisanie informacji o zgłoszonym wyjątku.
```

Po wpisaniu łańcucha znaków niereprezentującego liczby całkowitej otrzymujemy komunikat:



```
run:
Proszę podać piwerwszą liczbę:1a
null
```

W przypadku podania drugiej liczby zerowej następuje próba podzielenia liczby przez zero i otrzymujemy komunikat na skutek przechwycenia wyjątku przez system.



```
run:
Proszę podać piwerwszą liczbę:1
Proszę podać drugą liczbę:0
/ by zero
```

Obsługa wyjątku dla liczb rzeczywistych

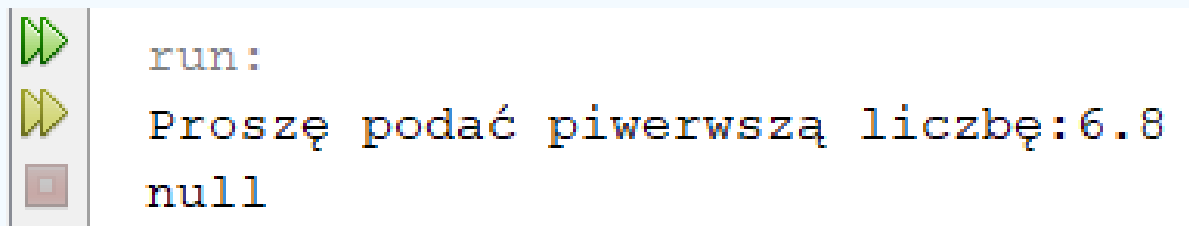
Jak widzimy, zgodnie z przewidywaniem przechwytywane są obydwa wyjątki. Spróbujmy sprawdzić działanie programu dla liczb rzeczywistych. W tym wypadku napisany program dokonujący podzielenia dwóch liczb rzeczywistych nie obsługuje wyjątku powstania nadmiaru zmiennoprzecinkowego i zostaje dokonany wydruk informujący użytkownika o powstałej sytuacji. Przedstawiony poniżej program ilustruje zachowanie się programu i systemu.

```
Scanner in = new Scanner(System.in) ;
try
// Instrukcja umożliwiająca wyznaczenie kodu
// mogącego doprowadzić do zgłoszenia wyjątku.
{   System.out.print
    ("Proszę podać piwerwszą liczbę:");
double a= in.nextDouble();
System.out.print
    ("Proszę podać drugą liczbę:");
    double b= in.nextDouble();
// Wczytanie dwóch liczb rzeczywistych.
double c = a/b;
// Dzielenie liczb rzeczywistych.
    System.out.printf("%g/%g=%g\n",a,b,c);
// Wypisanie wyników dzielenia.
}
```

Przechwycenie wyjątków

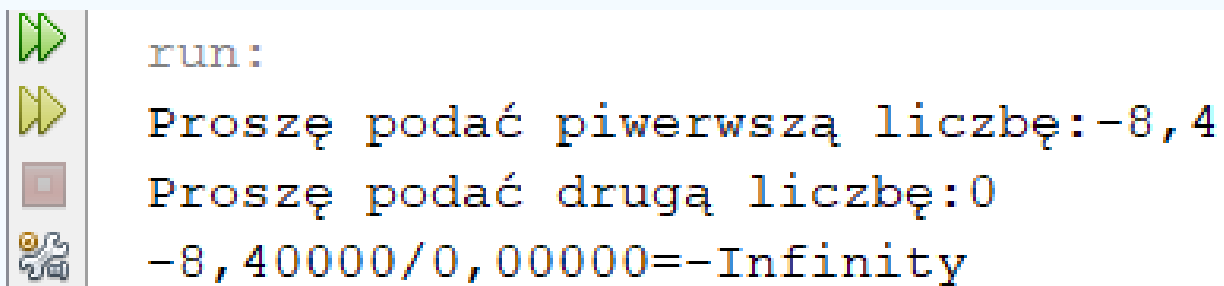
```
    catch (ArithmeticException ex) {  
// Blok przechwytyjący wyjątek.  
        System.out.println(ex.getMessage());  
    }  
// Wypisanie informacji o zgłoszonym wyjątku.  
    catch (Exception ex) {  
// Blok przechwytyjący wyjątek.  
        System.out.println(ex.getMessage());  
    }  
// Wypisanie informacji o zgłoszonym wyjątku.
```

Podanie liczby z kropką powoduje błąd w polskim interfejsie NetBeans IDE.



```
run:
Proszę podać piwerwszą liczbę:6.8
null
```

W przypadku próby podzielenia liczby ujemnej przez zero zostaje zamieszczony wydruk o nieskończoności.



```
run:
Proszę podać piwerwszą liczbę:-8,4
Proszę podać drugą liczbę:0
-8,40000/0,00000=-Infinity
```

Jeżeli chcemy zaradzić zaistniałej sytuacji możemy napisać metodę rozpoznającą błąd i zgłaszającą własny wyjątek. Poniżej prezentowany program prezentuje zgłoszenie własnego wyjątku w metodzie dokonującej podzielenia liczb rzeczywistych.

Zgłaszanie własnego wyjątku

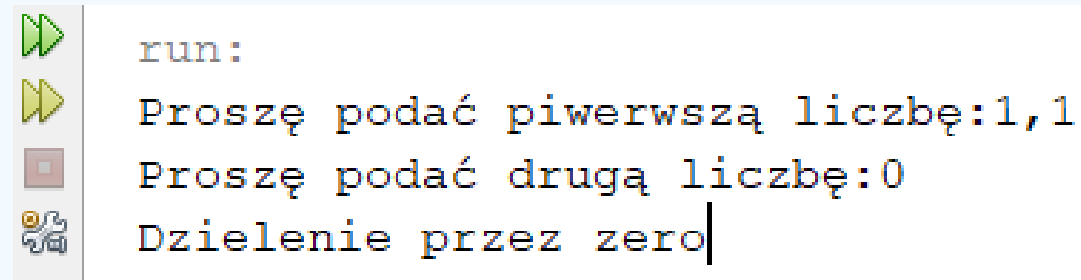
```
class Algorytm{ // Klasa Algorytm
    public double dzielenie(double a, double b)
// Metoda dzielenie.
    {
        try{
            if(b == 0) throw new ArithmeticException
                ("Dzielenie przez zero");
// Rozpoznanie błędu i wyrzucenie wyjątku.
        }
        catch( ArithmeticException e){
            throw e;
        }
        return a/b;
// Zwrócenie wyniku dzielenia.
    }
}
```

```
Scanner in = new Scanner(System.in);
try{
// Instrukcja umożliwiająca wyznaczenie kodu
// mogącego doprowadzić do zgłoszenia wyjątku.
    System.out.print
        ("Proszę podać piwerwszą liczbę:");
    double a= in.nextDouble();
    System.out.print
        ("Proszę podać drugą liczbę:");
    double b= in.nextDouble();
// Wczytanie dwóch liczb całkowitych.
    Algorytm at = new Algorytm();
    double c = at.dzielenie(a, b);
// Dzielenie liczb całkowitych.
    System.out.printf("%g/%g=%g%n",a,b,c);
// Wypisanie wyników dzielenia.
```

Przechwycenie wyjątków

```
        catch (ArithmeticException ex) {  
// Blok przechwytyjący wyjątek.  
        System.out.println(ex.getMessage());  
// Wypisanie informacji o zgłoszonym wyjątku.  
        catch (Exception ex) {  
// Blok przechwytyjący wyjątek.  
        System.out.println(ex.getMessage());  
// Wypisanie informacji o zgłoszonym wyjątku.
```

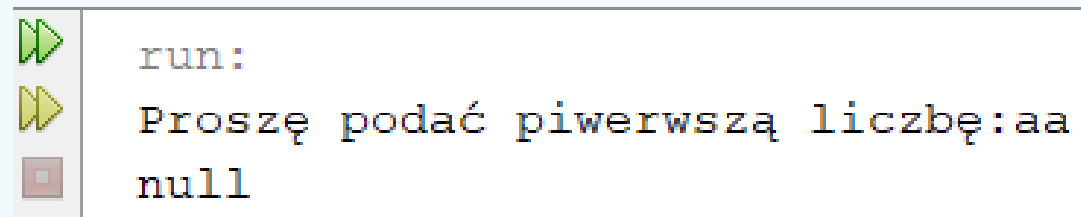
Jak widzimy zgłoszony przez nas wyjątek w metodzie dokonującej dzielenia liczb rzeczywistych działa poprawnie i przy próbie podzielenia przez zero zostaje wyświetlony komunikat podany w instrukcji wyrzucającej wyjątek jak na rysunku poniżej.



The screenshot shows a Java IDE console with a light yellow background. On the left, there are four icons: a green double arrow (run), a yellow double arrow (debug), a red square (stop), and a blue bug icon (exception). The text in the console is as follows:

```
run:
Proszę podać piwerwszą liczbę:1,1
Proszę podać drugą liczbę:0
Dzielenie przez zero
```

Ponieważ system przeszukuje listę bloków catch w celu znalezienia kodu obsługującego dany wyjątek w przypadku podania łańcucha nie reprezentującego liczby rzeczywistej również następuje zgłoszenie błędu przedstawionego na rysunku.



The screenshot shows a Java IDE console with a light yellow background. On the left, there are three icons: a green double arrow (run), a yellow double arrow (debug), and a red square (stop). The text in the console is as follows:

```
run:
Proszę podać piwerwszą liczbę:aa
null
```

Wynik obliczeń dla wprowadzonych danych






```
run:  
Proszę podać piwerwszą liczbę:8,8  
Proszę podać drugą liczbę:2,2  
8,80000/2,20000=4,00000
```

Klauzula throws

Jeżeli metoda może zgłosić wyjątek, z którym nie potrafi sobie poradzić, należy wtedy umieścić informacje o tym wyjątku w definicji. Wtedy program wywołujący metodę wie, jakich wyjątków może oczekiwać. W definicji informację o wyjątkach umieszcza się w klauzuli *throws*, która stanowi listę typów wyjątków zgłaszanych przez metodę. Nie trzeba w liście podawać wyjątków typu *Error* lub *RuntimeException*. Wszystkie pozostałe zgłaszane wyjątki trzeba wskazać jawnie, bo w przeciwnym wypadku kompilator zgłosi błąd. Ogólna postać deklaracji metody z klauzulą *throws* ma postać:

```
typ nazwa-metody(lista-parametrów) throws lista-wyjątków{  
    // kod metody  
}
```

```
public class C5e {  
    static void throwOne()  
        throws IllegalAccessException{  
        System.out.println("Wewnatrz throwone");  
        throw new IllegalAccessException("demo");  
    }  
    public static void main(String[] args) {  
        try{  
            throwOne();  
        }  
        catch (IllegalAccessException e){  
            System.out.println("Złapano - "+e);  
        }  
    }  
}
```



run:



Wewnatrz throwone



Złapano - java.lang.IllegalAccessException: demo

Niestandardowa klasa wyjątków

Niestandardowe klasy wyjątków są na ogół implementowane jeżeli powstające błędy są związane z klasą reprezentującą dany obiekt.

Rozpatrzmy zadanie rozwiązywania układu równań liniowych przedstawionego w rozdziale 3:

$$\begin{cases} a_{00}x_0 + a_{01}x_1 = b_0 \\ a_{10}x_0 + a_{11}x_1 = b_1 \end{cases}$$

Do rozwiązywania układu równań zastosowano metodę wyznacznikową, w której następuje podzielenie wyznaczników i powstający błąd jest związany z dwoma przypadkami:

- Wszystkie wyznaczniki są równe zero i układ równań ma nieskończenie wiele rozwiązań.
- Tylko wyznacznik główny jest równy zero, pozostałe wyznaczniki są różne od zera.

Do zaprogramowania metody znajdowania rozwiązania dwóch równań liniowych z obsługą błędów potrzebujemy własnej klasy obsługi błędów gdy wszystkie wyznaczniki są równe zero. Klasa obsługi błędu, gdy wszystkie wyznaczniki są równe zero jest przedstawiona poniżej:

Tworzenie własnej podklasy wyjątków

Ponieważ wbudowane wyjątki dotyczą tylko typowych błędów, to w przypadku wystąpienia takiej potrzeby w projektowaniu aplikacji będzie trzeba zdefiniować własne wyjątki związane z sytuacjami z sytuacjami specyficznymi dla danej aplikacji. Podczas tworzenie własnych podklas wystarczy rozszerzyć klasę *Exception* (która jest podklasą *Throwable*). Poniższy przykład deklaruje nową podklasę klasy *Exception* i używa jej do zasygnalizowania błędów w trakcie wykonywania metody rozwiązywania układu równań liniowych (układ równań nieoznaczony albo sprzeczny). Podklasy przysłaniają metodę *toString()*, aby spowodować wyświetlenie innego komunikatu przez instrukcję *println()*.

```
package c5h;
import java.util.*;
class NumbersZeroException extends Exception{
    private String s;
    NumbersZeroException(String s) {
        this.s =s;
    }
    public String toString() {
        return s;
    }
}
```

```
class Algorytm{
// Klasa algorytm.
    public void rozwiazanie(double [][] a, double [] b)
        throws NumbersZeroException
// Metoda rozwiązywania układów równań liniowych
// wyrzucająca wyjątki.
    { double W = a[0][0]*a[1][1] - a[0][1]*a[1][0];
      double Wx = b[0]*a[1][1] - a[0][1]*b[1];
      double Wy = a[0][0]*b[1] - b[0]*a[1][0];
// Wyznaczenie wyznaczników układu zapisanego macierzowo.
```

```
    if(W == 0 && Wx==0)
        throw new NumbersZeroException
            ("Układ równań jest nieznaczony");
// Wyrzucanie własnego wyjątku wszystkich liczb zerowych.
    if (W == 0)
        throw new NumbersZeroException
            ("Układ równań jest sprzeczny");
// Wyrzucenie wyjątku dzielenia przez zero.
    b[0]=Wx/W;
    b[1]=Wy/W;
// Zapisanie obliczonego rozwiązania.
}
}
```

```
public class C5d {  
    public static void main(String[] args) {  
        Scanner in = new Scanner(System.in);  
        try{  
            double [][] a= new double[2][2];  
            // Deklaracja macierzy A.  
            double[] b=new double[2];  
            // Deklaracja wektora b.  
            for(int i=0; i<2; i++){  
                for(int j=0; j<2; j++){  
                    System.out.printf  
                        ("Podaż a[%d][%d]:",i,j);  
                    a[i][j]= in.nextDouble();  
                }  
            }  
            // Wczytanie elementów macierzy A.
```

```
public class C5d {  
    public static void main(String[] args) {  
        Scanner in = new Scanner(System.in);  
        try{  
            double [][] a= new double[2][2];  
            // Deklaracja macierzy A.  
            double[] b=new double[2];  
            // Deklaracja wektora b.  
            for(int i=0; i<2; i++){  
                for(int j=0; j<2; j++){  
                    System.out.printf  
                        ("Podaż a[%d][%d]:",i,j);  
                    a[i][j]= in.nextDouble();  
                }  
            }  
            // Wczytanie elementów macierzy A.
```

```
        System.out.printf("Podaj b[%d]:", i);  
// Wczytanie elementów wektora b.  
        b[i] = in.nextDouble();  
    }  
    Algorytm at = new Algorytm();  
    at.rozwiazanie(a, b);  
    System.out.printf("x[0]=%g%n", b[0]);  
    System.out.printf("x[1]=%g%n", b[1]);  
}  
    catch (NumbersZeroException ex)  
// Przechwycenie wyjątku dzielenia przez zero.  
        { System.out.println(ex); }  
    catch (Exception ex)  
// Przechwycenie pozostałych wyjątków.  
        { System.out.println(ex.getMessage()); }  
}}
```

Wynik działania programu przedstawiono na rysunkach

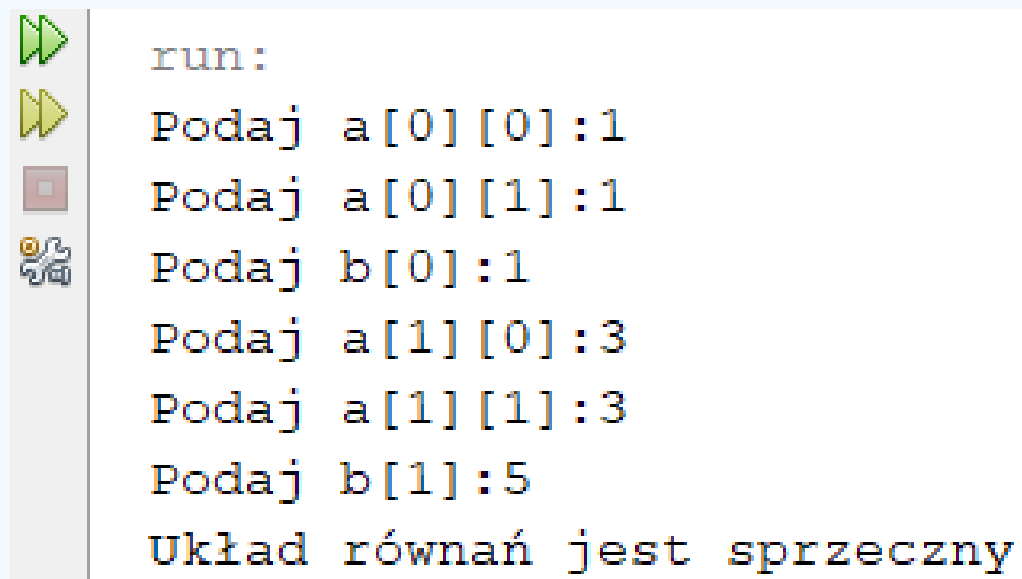


```
run:
Podaj a[0][0]:2
Podaj a[0][1]:1
Podaj b[0]:3
Podaj a[1][0]:1
Podaj a[1][1]:2
Podaj b[1]:3
x[0]=1,00000
x[1]=1,00000
```



```
run:
Podaj a[0][0]:1
Podaj a[0][1]:1
Podaj b[0]:1
Podaj a[1][0]:3
Podaj a[1][1]:3
Podaj b[1]:3
Układ równań jest nieznaczony
```

Wynik działania programu przedstawiono na rysunkach



```
run:  
Podaj a[0][0]:1  
Podaj a[0][1]:1  
Podaj b[0]:1  
Podaj a[1][0]:3  
Podaj a[1][1]:3  
Podaj b[1]:5  
Układ równań jest sprzeczny
```

Uwaga. Przedstawione rozumowanie nie przenosi się automatycznie na wyższe wymiary i wymaga zastosowania aparatu analizy numerycznej do algorytmu rozwiązywania układów równań liniowych.

Użycie bloku finally

Blok finally jest wykonywane niezależnie od tego, czy podczas wykonywania instrukcji w bloku *try* został wywołany jakiś wyjątek, czy też nie. Blok *finnally* jest blokiem opcjonalnym jeśli zdefiniowany jest blok *catch* przechwytyjący wyjątki. W przypadku nie zdefiniowania bloków przechwytyjących wyjątki jest blokiem wymaganym. Wykorzystanie bloku *finnally* (bloku czyszczącego) jest przydatne w operacjach na plikach. Przedstawiony poniżej program wczytuje cały plik tekstowy, sprawdza poprawność jego otwarcia i w przypadku wczytania pliku wyświetla jego zawartość na monitorze. Blok *finnally* został wykorzystany do zamknięcia pliku w przypadku jego otwarcia.

```
import java.io.*;
public class C5f {
    private static final String PLIK = "dane.txt";
    public static void main(String[] args) {
        BufferedReader br = null;
        FileReader fr = null;
        try{
            //br = new BufferedReader(new FileReader(PLIK));
            fr = new FileReader(PLIK);
            br = new BufferedReader(fr);
            String s;
            while ((s = br.readLine()) != null) {
                System.out.println(s);
            }
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

```
    } finally {  
        try{  
            if (br != null)  
br.close();  
            if (fr != null)  
fr.close();  
        } catch (IOException ex) {  
            ex.printStackTrace();  
        }  
    }  
}  
}}
```

	run:
	Pierwsza linia
	Druga linia
	Trzecia Linia

Przechwycenie wyjątku IOException przez system i wyświetlenie komunikatu o błędzie

Wyjście - C5f (run) ×

```
run:
java.io.FileNotFoundException: dane.txt (Nie można odnaleźć określonego pliku)
    at java.io.FileInputStream.open0(Native Method)
    at java.io.FileInputStream.open(FileInputStream.java:195)
    at java.io.FileInputStream.<init>(FileInputStream.java:138)
    at java.io.FileInputStream.<init>(FileInputStream.java:93)
    at java.io.FileReader.<init>(FileReader.java:58)
    at c5f.C5f.main(C5f.java:10)
BUILD SUCCESSFUL (total time: 0 seconds)
```